

Authentication, Authorization, OAuth2, OIDC, PKCE, RBAC, and Spring Security

This guide is written for a beginner who wants to learn from scratch and understand how the concepts fit together in Spring applications and in modern security architecture generally.

It covers:

- Authentication vs authorization
- Common security and login flows
- Session/cookie auth vs token auth
- OAuth 2.0 and OpenID Connect
- Access token, refresh token, and ID token
- PKCE in detail
- RBAC
- Auth server vs resource server
- How all of this maps to Spring Security and Spring Authorization Server
- Diagram-based explanations of the main flows

One correction first:

- `RABC` is almost certainly `RBAC` = `Role-Based Access Control`
-

1. Big Picture

When people say "security" in an application, they usually mix several different problems together:

1. How do we know who the user is?
2. How do we check what the user can do?
3. How do we keep credentials safe?
4. How do clients call APIs securely?
5. How do different systems trust each other?
6. How do we support login without exposing passwords to every application?

Modern systems solve these with different building blocks:

- `Authentication` proves identity
- `Authorization` decides permissions
- `Session` or `Token` carries authenticated state
- `OAuth 2.0` delegates access
- `OpenID Connect (OIDC)` adds identity on top of OAuth 2.0
- `PKCE` protects public clients in authorization code flow

- **RBAC** organizes permissions by role
 - **Authorization Server** issues tokens
 - **Resource Server** validates tokens and protects APIs
-

2. Authentication vs Authorization

These are the first two things to understand clearly.

Authentication

Authentication answers:

- "Who are you?"

Examples:

- Username + password login
- Login with Google
- OTP login
- Biometrics
- Client certificate
- API key identifying an application

If authentication succeeds, the system knows the subject:

- user identity
- client identity
- service identity

Authorization

Authorization answers:

- "What are you allowed to do?"

Examples:

- Can user access `/admin` ?
- Can user delete invoices?
- Can service A call service B?
- Can mobile app read profile data?

Simple example

Imagine a banking app:

- User logs in with email and password
- Server verifies password

- This is `authentication`
- User tries to open admin dashboard
- Server checks whether user has admin permission
- This is `authorization`

Spring Security view

In Spring Security:

- authentication creates an `Authentication` object
- that object contains the principal and granted authorities
- authorization checks those authorities at request or method level

Example:

```
@Bean
SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/admin/**").hasRole("ADMIN")
            .requestMatchers("/profile/**").authenticated()
            .anyRequest().permitAll()
        )
        .formLogin(Customizer.withDefaults());

    return http.build();
}
```

Meaning:

- public endpoints are open
- `/profile/**` needs login
- `/admin/**` needs the `ADMIN` role

3. Core Security Terms

Before going deeper, learn these terms:

Principal

The identity being authenticated.

Examples:

- a user

- a service account
- a client application

Credential

Something used to prove identity.

Examples:

- password
- OTP
- private key
- client secret

Session

Server-side authenticated state usually linked by a browser cookie.

Token

A value given to a client that can later be presented as proof of authorization or identity.

Examples:

- access token
- refresh token
- ID token

Claim

A statement inside a token.

Examples:

- `sub` = subject
- `exp` = expiry time
- `scope` = allowed scopes
- `roles` = roles

Scope

A named permission requested by a client and granted by the authorization server.

Examples:

- `profile`
- `email`
- `read:orders`
- `write:orders`

Role

A business-level grouping of permissions.

Examples:

- ADMIN
- MANAGER
- USER

Authority

In Spring, this is a general permission string used during authorization.

Examples:

- ROLE_ADMIN
 - invoice:read
 - invoice:write
-

4. Main Security Models

There are two broad models you will see most often in Spring apps.

A. Session-Based Authentication

Common in:

- traditional web apps
- Spring MVC apps
- admin dashboards

How it works:

1. User logs in with username and password
2. Server verifies credentials
3. Server creates a session
4. Session ID is stored in a cookie
5. Browser sends the cookie on later requests

Advantages:

- simple for browser apps
- easy logout and invalidation
- no token handling in frontend code

Tradeoffs:

- server must keep session state

- less ideal for distributed APIs unless session replication is used

B. Token-Based Authentication

Common in:

- REST APIs
- SPAs
- mobile apps
- microservices

How it works:

1. Client gets a token
2. Client sends token on API requests
3. API validates the token

Advantages:

- stateless APIs
- fits mobile and SPA clients
- works well across services

Tradeoffs:

- token lifetime and revocation need careful design
 - frontend storage must be handled carefully
-

5. Common Security Flows

This section goes beyond Spring Security and explains common real-world flows.

5.1 Form Login Flow

Best for:

- server-rendered web apps
- internal business systems
- simple Spring MVC apps

Flow:

```
Browser -> GET /login
Server -> login page
Browser -> POST username/password
Server -> verify credentials
Server -> create session + set cookie
Browser -> sends session cookie on future requests
```

Diagram:

```
sequenceDiagram
    participant U as User Browser
    participant A as App Server
    U->>A: GET /login
    A-->>U: Login page
    U->>A: POST username/password
    A->>A: Validate credentials
    A-->>U: Set session cookie
    U->>A: GET /profile with cookie
    A-->>U: Profile page
```

Spring use:

```
http.formLogin(Customizer.withDefaults());
```

5.2 HTTP Basic Authentication

Best for:

- simple internal APIs
- quick testing
- service calls in trusted environments

Flow:

```
Client -> Authorization: Basic base64(username:password)
Server -> validate credentials for every request
```

Notes:

- not user-friendly for browsers
- must use HTTPS
- often replaced by bearer tokens in modern APIs

Spring use:

```
http.httpBasic(Customizer.withDefaults());
```

5.3 API Key Authentication

Best for:

- simple service integrations
- internal platform access
- low-complexity machine clients

Flow:

```
Client -> X-API-Key: abc123  
Server -> looks up key and grants access
```

Notes:

- simple, but weaker and less expressive than OAuth2
- usually authenticates the client/application, not the end user
- does not naturally express delegated user consent

5.4 Bearer Token Flow

Best for:

- stateless APIs
- mobile clients
- microservices

Flow:

```
Client gets access token  
Client -> Authorization: Bearer <access_token>  
Resource Server -> validates token  
Resource Server -> returns protected data
```

Spring use:

```
http.oauth2ResourceServer(oauth2 -> oauth2.jwt());
```

5.5 OAuth 2.0 Authorization Code Flow

Best for:

- web apps
- apps using SSO
- apps integrating with Google, GitHub, Okta, Keycloak, Auth0

High-level idea:

- the app does not directly collect or verify the user's password against the identity provider
- user authenticates at the authorization server
- the app receives an authorization code
- the app exchanges the code for tokens

Flow summary:

1. Client redirects user to authorization server
2. User logs in there
3. Authorization server sends authorization code back
4. Client exchanges code for access token, and maybe refresh token / ID token

Diagram:

```
sequenceDiagram
    participant U as User
    participant C as Client App
    participant AS as Authorization Server
    participant RS as Resource Server
    U->>C: Open app
    C-->>U: Redirect to AS
    U->>AS: Login and consent
    AS-->>U: Redirect back with authorization code
    U->>C: Return with code
    C->>AS: Exchange code for tokens
    AS-->>C: Access token (+ refresh token, maybe ID token)
    C->>RS: API request with access token
    RS-->>C: Protected resource
```

Spring use:

```
http.oauth2Login(Customizer.withDefaults());
```

5.6 Authorization Code Flow with PKCE

Best for:

- SPA clients
- mobile apps

- desktop apps
- any public client that cannot safely store a client secret

This is now one of the most important flows in modern authentication.

Detailed PKCE section is below.

5.7 Client Credentials Flow

Best for:

- machine-to-machine communication
- backend services
- batch jobs

Flow:

```
Client authenticates with client_id + client_secret
Auth Server issues access token
Client calls API using that token
```

Diagram:

```
sequenceDiagram
    participant C as Service Client
    participant AS as Authorization Server
    participant RS as Resource Server
    C->>AS: Token request with client credentials
    AS-->>C: Access token
    C->>RS: API request with bearer token
    RS-->>C: Protected resource
```

Important:

- no end user is involved
- this is for client/application authorization

5.8 Device Authorization Flow

Best for:

- smart TVs
- consoles
- CLI tools
- devices with poor keyboard/input

Flow:

1. Device requests device code and user code
2. Device shows code and verification URL
3. User opens URL on another device
4. User logs in and approves access
5. Original device polls token endpoint
6. Device receives access token

5.9 Refresh Token Flow

Best for:

- renewing short-lived access tokens
- avoiding repeated login prompts

Flow:

```
Client has refresh token
Access token expires
Client sends refresh token to token endpoint
Auth Server returns new access token
```

Important:

- refresh token goes to the authorization server
- not to the resource server

5.10 OIDC Login Flow

OIDC is OpenID Connect.

It builds on OAuth 2.0 and adds identity.

What OAuth 2.0 gives:

- delegated authorization

What OIDC adds:

- identity layer
- ID token
- standard user info handling

This is why "Login with Google" is usually OIDC, not plain OAuth2.

5.11 SAML SSO Flow

You asked for more than Spring Security only, so this is worth mentioning.

Best for:

- enterprise SSO

- older corporate identity systems

Concept:

- Identity Provider authenticates user
- Service Provider trusts signed assertions

Spring applications can integrate with SAML, but for new greenfield apps OAuth2/OIDC is more common.

6. OAuth 2.0 From Scratch

OAuth 2.0 is often misunderstood as "login". That is incomplete.

OAuth 2.0 is mainly an authorization framework for delegated access.

Example:

- you allow a printing app to access your Google Drive files
- the printing app should not know your Google password
- Google issues a token with specific permissions

That is OAuth2.

OAuth2 actors

The official OAuth2 model defines these main roles:

- Resource Owner
- Client
- Authorization Server
- Resource Server

Resource Owner

Usually the user who owns the data.

Client

The app requesting access.

Examples:

- web app
- SPA
- mobile app
- backend service

Authorization Server

The system that authenticates users or clients and issues tokens.

Examples:

- Spring Authorization Server
- Keycloak
- Okta
- Auth0
- Google Identity

Resource Server

The API that hosts protected resources and validates access tokens.

OAuth2 key endpoints

Common endpoints on an authorization server:

- authorization endpoint
- token endpoint
- JWK set endpoint
- introspection endpoint
- revocation endpoint

If OIDC is enabled, also:

- userinfo endpoint
 - OIDC discovery endpoint
-

7. OpenID Connect From Scratch

OpenID Connect adds identity to OAuth 2.0.

If OAuth2 says:

- "an app can access a resource"

OIDC says:

- "and here is the authenticated identity of the user"

Why OIDC exists

Plain OAuth2 is about access delegation. Applications also needed a standard way to:

- authenticate users
- receive identity claims
- do SSO

OIDC solves that with:

- ID token
- standard claims
- userinfo endpoint

- discovery metadata

Common OIDC scopes

- `openid` required for OIDC
- `profile`
- `email`
- `phone`
- `address`

If you request `openid`, the system knows this is OIDC, not only OAuth2.

8. Access Token vs Refresh Token vs ID Token

This is one of the most important sections for beginners.

8.1 Access Token

Purpose:

- used to call protected APIs

Audience:

- resource server

What it says:

- the client has permission to access some resource with certain scopes/claims

Example:

```
GET /api/orders
Authorization: Bearer eyJ...
```

Typical contents:

- issuer
- subject
- audience
- expiry
- scopes
- authorities or roles

Important:

- access token is for `authorization`

- it is usually short-lived
- it may be JWT or opaque

8.2 Refresh Token

Purpose:

- used to obtain a new access token

Audience:

- authorization server only

Important:

- never send refresh token to an API/resource server
- usually longer-lived than access token
- must be protected carefully

8.3 ID Token

Purpose:

- used to represent the authenticated identity of the user in OIDC

Audience:

- client application

What it commonly contains:

- `iss` issuer
- `sub` subject
- `aud` audience
- `exp` expiration
- `iat` issued at
- authentication-related claims
- profile claims depending on scopes

Important:

- ID token is for `authentication`
- access token is for `API access`
- do not confuse them

Quick comparison table

Token	Main purpose	Sent to	Typical use
Access token	API authorization	Resource Server	Call protected API
Refresh token	Renew access token	Authorization Server	Get new access token
ID token	User identity	Client	Know who logged in

Common beginner mistake

Wrong:

- using ID token as bearer token for APIs

Correct:

- use access token for APIs
 - use ID token to understand authenticated identity in OIDC login
-

9. PKCE in Detail

PKCE stands for:

- Proof Key for Code Exchange

It is defined in RFC 7636.

PKCE was created to protect the authorization code flow, especially for public clients.

Why PKCE is needed

Public clients cannot safely keep a client secret.

Examples:

- browser-based SPA
- mobile app
- desktop app

If an attacker steals the authorization code, they may try to exchange it for tokens.

PKCE reduces that risk by binding the authorization request and token exchange together.

PKCE components

The client generates:

- `code_verifier` = high-entropy random secret
- `code_challenge` = derived from `code_verifier`

Usually:

- `code_challenge = BASE64URL(SHA256(code_verifier))`
- `method = S256`

PKCE flow step by step

1. Client generates a random `code_verifier`
2. Client creates `code_challenge` from it
3. Client sends authorization request with `code_challenge`
4. User authenticates at authorization server
5. Authorization server returns authorization code
6. Client sends code + original `code_verifier` to token endpoint
7. Authorization server recomputes the challenge
8. If they match, tokens are issued

PKCE diagram

```
sequenceDiagram
    participant U as User
    participant C as Public Client
    participant AS as Authorization Server
    C->>C: Generate code_verifier
    C->>C: Derive code_challenge
    C->>AS: Authorization request + code_challenge
    U->>AS: Login and consent
    AS-->>C: Authorization code
    C->>AS: Token request + code + code_verifier
    AS->>AS: Validate verifier against challenge
    AS-->>C: Access token (+ refresh token / ID token)
```

Why PKCE is important

Without PKCE:

- stolen authorization code may be usable by attacker

With PKCE:

- attacker also needs the original `code_verifier`

When to use PKCE

Use PKCE for:

- SPA

- mobile apps
- desktop apps
- native apps
- generally all authorization code flows unless you have a strong reason not to

Modern best practice is:

- Authorization Code + PKCE

PKCE and Spring Security

Spring Security supports PKCE for OAuth2 clients.

In practice:

- if the client is public or configured appropriately, Spring can handle PKCE automatically in OAuth2 client flows

PKCE vs client secret

They solve different problems:

- `client_secret` proves client identity for confidential clients
- `PKCE` proves continuity between auth request and token exchange

A confidential client may still use PKCE.

Important beginner takeaway

If you are building:

- web backend app with secure server secret: Authorization Code is common, PKCE is still a good idea
 - SPA or mobile app: Authorization Code with PKCE is the modern standard
-

10. Auth Server vs Resource Server vs Client

These roles are often mixed up. Keep them separate.

Client

The app requesting access.

Examples:

- Spring MVC app
- Angular frontend
- Android app
- backend service

Authorization Server

The server that:

- authenticates users or clients
- obtains consent if needed
- issues tokens
- exposes token/authorization/JWK endpoints

Examples:

- Spring Authorization Server
- Keycloak
- Okta
- Auth0

Resource Server

The API that:

- receives bearer access tokens
- validates them
- returns protected resources

Example:

- your Spring Boot REST API

Diagram

```
flowchart LR
    U[User] --> C[Client Application]
    C --> AS[Authorization Server]
    C --> RS[Resource Server API]
    AS --> C
    RS --> C
```

Real example

- User logs into an Angular app
- Angular app redirects to Auth Server
- Auth Server returns tokens
- Angular app calls Spring Boot API with access token
- Spring Boot API is the Resource Server

11. RBAC From Scratch

RBAC means:

- Role-Based Access Control

Instead of assigning every permission directly to every user, the system assigns permissions to roles, and users receive roles.

Simple RBAC structure

```
User -> Role -> Permissions
```

Example:

```
Alice -> ADMIN -> create_user, delete_user, read_report
Bob    -> USER  -> read_own_profile
Sara   -> AUDITOR -> read_report
```

Why RBAC is useful

- easier to manage
- maps to business structure
- reduces duplication
- clearer than ad-hoc permission checks

Example roles

- ADMIN
- MANAGER
- ACCOUNTANT
- SUPPORT
- USER

Example permissions

- invoice:read
- invoice:write
- invoice:delete
- user:create
- user:disable

Roles vs permissions

Use:

- `roles` for broad business categories
- `permissions/authorities` for precise access rules

RBAC in Spring

Spring often uses:

- roles as `ROLE_ADMIN`
- authorities as custom strings like `invoice:read`

Examples:

```
.requestMatchers("/admin/**").hasRole("ADMIN")
.requestMatchers(HttpMethod.GET, "/invoices/**").hasAuthority("invoice:read")
.requestMatchers(HttpMethod.POST, "/invoices/**").hasAuthority("invoice:write")
```

Method security:

```
@PreAuthorize("hasRole('ADMIN')")
public void createUser() {}

@PreAuthorize("hasAuthority('invoice:write')")
public void createInvoice() {}
```

Important Spring detail

`hasRole("ADMIN")` usually expects authority:

- `ROLE_ADMIN`

That prefix behavior confuses many beginners.

12. Authentication and Authorization in Spring Security

Spring Security is the main Spring project for application security.

At a high level it helps with:

- authentication
- authorization
- session security
- OAuth2 client

- resource server support
- OIDC login
- method security
- CSRF protection
- password handling

12.1 Authentication in Spring Security

Authentication usually involves:

- login request enters filter chain
- credentials are parsed
- an `AuthenticationManager` authenticates
- result becomes an authenticated `Authentication`
- security context is populated

Conceptually:

```
Request -> Filter -> AuthenticationManager -> UserDetailsService / Provider  
-> Authenticated principal -> SecurityContext
```

12.2 Authorization in Spring Security

Authorization can happen at:

- request level
- method level

Request level:

```
http.authorizeHttpRequests(auth -> auth  
    .requestMatchers("/admin/**").hasRole("ADMIN")  
    .anyRequest().authenticated()  
);
```

Method level:

```
@EnableMethodSecurity  
@Configuration  
class SecurityConfig {  
}
```

```
@Service
class InvoiceService {
    @PreAuthorize("hasAuthority('invoice:read')")
    public Invoice getInvoice(Long id) {
        return null;
    }
}
```

12.3 Password handling

Never store plain text passwords.

Use password hashing via:

- `PasswordEncoder`

Common example:

```
PasswordEncoder encoder = new BCryptPasswordEncoder();
```

12.4 Session auth in Spring

Useful for:

- traditional browser apps

Typical config:

```
@Bean
SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .authorizeHttpRequests(auth -> auth
            .anyRequest().authenticated()
        )
        .formLogin(Customizer.withDefaults());

    return http.build();
}
```

12.5 JWT bearer auth in Spring

Useful for:

- REST APIs
- microservices

Example:

```
@Bean
SecurityFilterChain apiSecurity(HttpSecurity http) throws Exception {
    http
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/public/**").permitAll()
            .anyRequest().authenticated()
        )
        .oauth2ResourceServer(oauth2 -> oauth2.jwt());

    return http.build();
}
```

Configuration example:

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: https://auth.example.com
```

12.6 OAuth2 login in Spring

Useful for:

- social login
- SSO
- enterprise login through identity providers

Example:

```
@Bean
SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .authorizeHttpRequests(auth -> auth.anyRequest().authenticated())
        .oauth2Login(Customizer.withDefaults());

    return http.build();
}
```


12.7 Resource Server in Spring

A Spring app becomes a resource server when it validates bearer access tokens and protects APIs.

Spring supports:

- JWT validation
- opaque token introspection

12.8 Spring Authorization Server

If you want to build your own auth server in Spring, use:

- `Spring Authorization Server`

It provides:

- authorization endpoint
- token endpoint
- client authentication
- refresh tokens
- JWT support
- OIDC support
- JWK set endpoint
- introspection
- revocation

Use it when:

- you need your own central identity provider
 - multiple apps share one login system
 - you need your own token issuance rules
-

13. Text Diagrams of Important Flows

Some learners understand better through simplified diagrams. Here are compact versions.

13.1 Session Login

```
[Browser]
  |
  | username/password
  v
[App Server]
  |
  | create session
  v
[Session Store / Memory]

Later:
[Browser] -- cookie --> [App Server]
```

13.2 Bearer Token API Flow

```
[Client] -- bearer token --> [Resource Server API]
                                |
                                | validate token
                                v
                                [Allow / Deny]
```

13.3 OAuth2 Authorization Code Flow

```
User -> Client App -> Authorization Server
                        <- redirect for login
User logs in at Authorization Server
Authorization Server -> Client App (authorization code)
Client App -> Authorization Server (exchange code)
Authorization Server -> Client App (tokens)
Client App -> Resource Server (access token)
```

13.4 Authorization Code + PKCE

```
Client generates code_verifier
Client derives code_challenge
Client -> Authorization Server (auth request + code_challenge)
User authenticates
Authorization Server -> Client (authorization code)
Client -> Authorization Server (code + code_verifier)
Authorization Server validates proof
Authorization Server -> Client (tokens)
```

13.5 Client Credentials

```
Service A -> Auth Server (client_id + client_secret)
Auth Server -> Service A (access token)
Service A -> Service B API (access token)
Service B API -> Service A (resource)
```

13.6 OIDC Login

```
User -> Client App
Client App -> Identity Provider
Identity Provider authenticates user
Identity Provider -> Client App (ID token + access token)
Client App knows who user is from ID token
```

14. Which Flow Should You Use?

This is the practical question.

Scenario: Traditional Spring MVC app

Use:

- form login
- sessions

Why:

- simplest
- natural browser fit

Scenario: REST API for mobile or SPA

Use:

- bearer access tokens
- resource server
- authorization code with PKCE for the frontend login side

Why:

- stateless API
- better separation

Scenario: "Login with Google"

Use:

- OAuth2 login with OIDC

Why:

- delegated authentication
- standard ID token support

Scenario: backend service calling another backend service

Use:

- client credentials

Why:

- no human user

Scenario: smart TV or CLI login

Use:

- device authorization flow

Scenario: one company identity system for many apps

Use:

- authorization server / identity provider
 - OIDC
 - Spring Authorization Server if building in Spring
-

15. Security Mistakes Beginners Often Make

These are worth remembering early.

Concept mistakes

- thinking authentication and authorization are the same thing
- thinking OAuth2 is only "login"
- thinking JWT automatically means secure design
- confusing access token and ID token

Implementation mistakes

- storing plain text passwords
- using long-lived access tokens without reason
- sending refresh token to API/resource server
- putting secrets in SPA/mobile source code
- skipping HTTPS
- using broad roles without fine-grained authority checks
- not validating token issuer/audience correctly
- exposing sensitive claims unnecessarily

PKCE mistakes

- not using PKCE for public clients
- using weak code verifier generation
- assuming PKCE replaces HTTPS

Spring mistakes

- forgetting `ROLE_` prefix behavior
- putting all authorization rules only in controllers
- disabling CSRF without understanding why
- writing custom authentication code before understanding built-in support

16. Recommended Learning Path for a Spring Beginner

Learn in this order:

1. Authentication vs authorization
2. Sessions, cookies, and form login
3. Password hashing and `PasswordEncoder`
4. Roles vs authorities
5. Request authorization in Spring Security
6. Method security with `@PreAuthorize`
7. JWT bearer token validation with resource server

8. OAuth2 actors and authorization code flow
9. OIDC and ID token
10. PKCE
11. Spring Authorization Server

If you try to start directly with OAuth2 and OIDC without understanding sessions, authentication, and authorization, everything feels more complicated than it really is.

17. Spring Mapping Cheat Sheet

Use this as a compact memory map.

Need	Spring feature
Browser login page	<code>formLogin()</code>
Simple auth prompt	<code>httpBasic()</code>
Protect REST API with bearer JWT	<code>oauth2ResourceServer().jwt()</code>
Login with Google/Okta	<code>oauth2Login()</code>
Request-level authorization	<code>authorizeHttpRequests(...)</code>
Method-level authorization	<code>@EnableMethodSecurity</code> , <code>@PreAuthorize</code>
Password hashing	<code>PasswordEncoder</code>
Build your own authorization server	Spring Authorization Server

18. Minimal Spring Examples

18.1 Basic form login

```
@Configuration
public class SecurityConfig {

    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception
    {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/public/**").permitAll()
                .anyRequest().authenticated()
            )
            .formLogin(Customizer.withDefaults());

        return http.build();
    }
}
```

18.2 Method security

```
@Configuration
@EnableMethodSecurity
class MethodSecurityConfig {
}
```

```
@Service
class ReportService {

    @PreAuthorize("hasRole('ADMIN')")
    public String adminReport() {
        return "secret";
    }
}
```

18.3 JWT resource server

```
@Configuration
public class ApiSecurityConfig {

    @Bean
    SecurityFilterChain apiSecurity(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/public/**").permitAll()
                .anyRequest().authenticated()
            )
            .oauth2ResourceServer(oauth2 -> oauth2.jwt());

        return http.build();
    }
}
```

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: https://auth.example.com
```

18.4 OAuth2 login

```
spring:
  security:
    oauth2:
      client:
        registration:
          google:
            client-id: your-client-id
            client-secret: your-client-secret
            scope: openid, profile, email
```



```
@Bean
SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .authorizeHttpRequests(auth -> auth.anyRequest().authenticated())
        .oauth2Login(Customizer.withDefaults());

    return http.build();
}
```

19. Official Sources

These are the primary references this guide is based on.

Spring docs

- Spring Security reference: <https://docs.spring.io/spring-security/reference/index.html>
- Spring Security authentication: <https://docs.spring.io/spring-security/reference/features/authentication/index.html>
- Spring Security authorization: <https://docs.spring.io/spring-security/reference/features/authorization/index.html>
- Spring Security authorization architecture: <https://docs.spring.io/spring-security/reference/servlet/authorization/architecture.html>
- Spring Security method security: <https://docs.spring.io/spring-security/reference/servlet/authorization/method-security.html>
- Spring Security OAuth2 overview: <https://docs.spring.io/spring-security/reference/servlet/oauth2/index.html>
- Spring Security OAuth2 login: <https://docs.spring.io/spring-security/reference/7.0/servlet/oauth2/login/index.html>
- Spring Security OAuth2 client authorization grants: <https://docs.spring.io/spring-security/reference/7.0/servlet/oauth2/client/authorization-grants.html>
- Spring Security resource server JWT: <https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/jwt.html>
- Spring Authorization Server reference: <https://docs.spring.io/spring-authorization-server/reference/index.html>
- Spring Authorization Server overview: <https://docs.spring.io/spring-authorization-server/reference/overview.html>

OAuth2 and OIDC standards

- OAuth 2.0 Framework RFC 6749: <https://www.rfc-editor.org/rfc/rfc6749>
- OAuth 2.0 Bearer Token Usage RFC 6750: <https://www.rfc-editor.org/rfc/rfc6750>

- PKCE RFC 7636: <https://www.rfc-editor.org/rfc/rfc7636>
- OAuth 2.0 for Native Apps RFC 8252: <https://www.rfc-editor.org/rfc/rfc8252>
- OpenID Connect Core 1.0: https://openid.net/specs/openid-connect-core-1_0.html

Access control references

- NIST RBAC project: <https://csrc.nist.gov/projects/role-based-access-control>
 - NIST RBAC paper: <https://www.nist.gov/publications/role-based-access-control-rbac-features-and-motivations>
-

20. Final Mental Model

If you remember only one summary, remember this:

- `Authentication` = who are you
 - `Authorization` = what can you do
 - `Session` = browser/server state after login
 - `Access token` = lets a client call APIs
 - `Refresh token` = gets a new access token
 - `ID token` = tells the client who the user is
 - `OAuth2` = delegated authorization
 - `OIDC` = identity on top of OAuth2
 - `PKCE` = protects authorization code flow, especially for public clients
 - `RBAC` = roles organize permissions
 - `Authorization Server` = issues tokens
 - `Resource Server` = protects APIs using tokens
 - `Spring Security` = secures your app
 - `Spring Authorization Server` = lets you build the auth server itself
-

21. What To Study Next

After reading this file, a good next sequence is:

1. Build one small Spring MVC app with `formLogin`
2. Add roles and `@PreAuthorize`
3. Build one REST API with `oauth2ResourceServer().jwt()`
4. Learn how JWT claims map to Spring authorities
5. Try `oauth2Login()` with Google or GitHub
6. Explore Spring Authorization Server

If you want, the next useful follow-up would be:

- a second document focused only on Spring Security architecture
- a step-by-step Spring Boot auth project
- or a Bangla + English beginner note version of this file